

A Worker Pool in Python

```
from concurrent.futures import ThreadPoolExecutor

def fetch(url: str) -> bytes:
    with urlopen(url, timeout=5) as response:
        return response.read()

with ThreadPoolExecutor(max_workers=8) as pool:
    for body in pool.map(fetch, urls):    # order preserved
        process(body)
```

The pool bounds concurrency; the map call bounds memory.

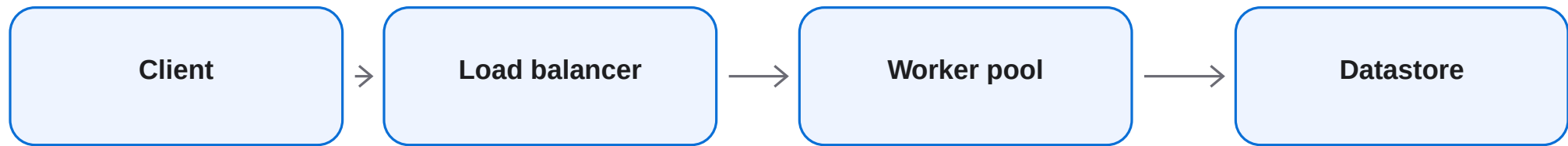
Deadlock, in Four Lines

```
thread A: lock(m1); lock(m2);    // holds m1, wants m2
thread B: lock(m2); lock(m1);    // holds m2, wants m1

fix: impose a global lock order, or use try_lock with backoff
```

Coffman conditions: mutual exclusion, hold and wait, no preemption, circular wait.

Request Pipeline



Back pressure propagates right to left.

Summary

- Bound concurrency explicitly; unbounded queues hide failure.
- Order your locks, or avoid holding two at once.
- Measure tail latency, not the mean.